

Scenario

Scenario A (creative): game patch / content release pipeline. A mid-size studio ships a content update. A feature request is approved by a producer and then splits into parallel work: art, code, audio and narrative. These join for integration and a build, then go to QA. Failed QA loops back to the responsible discipline. Then came creative-director sign-off, platform certification (Steam, PlayStation, Nintendo, each with its own API and rules), and release. It is approval-heavy, with lots of fork/join, rework loops, escalation when cert is rejected, and external systems (CI server, bug tracker, storefront).

Scenario B (logistics/analytics): cold-chain pharmaceutical shipment. A distributor ships temperature-sensitive vaccines. Order intake leads to a stock check, packing, carrier assignment, and customs clearance (external customs API). While the shipment is in transit, IoT temperature sensors emit events. A temperature excursion triggers new work: a quality investigation, a quarantine decision, and possibly a replacement shipment. Delivery confirmation ends the workflow, followed by analytics on delays per route and carrier. This one is event-driven and deadline-driven, and it is light on human creativity.

Here's how both plug into the same variation points:

Variation point	A	B
Assignment rule	By discipline role + lowest workload	By warehouse region / carrier availability
Approval chain	Lead → Producer → Creative Director	QA officer → Pharmacist → Compliance manager
External systems	CI build server, Steam/console cert	Customs API, carrier tracking, IoT sensors
Triggering events	Build failed, cert rejected	Temperature excursion, customs hold
Task add-ons	Priority (hotfix), audit	Security (controlled substance), audit, deadline
Rollback	Revert to pre-QA state after bad merge	Undo mis-assignment of carrier

Real-world comparisons:

- Jira's workflow engine
- ServiceNow
- Apache Airflow (a good fit for B)
- GitHub Actions (less relevant in this prac, but still want to note)
- NA10(ai heavy)
- make.com(monday uses this)

Patterns

Tiered set. Tier 1 gets me to 10, and each tier-1 pattern maps almost word for word to something in section 4 of the spec, which makes justifications easy.

Tier 1

Pattern	Problem	Notes
Composite	Workflows contain stages, tasks and sub-processes, nested arbitrarily	WorkflowNode (component), Task (leaf), Stage/SubProcess (composite)
Prototype	Many independent instances from one definition	WorkflowDefinition::createInstance() deep-clones WorkflowNode::clone()
Builder	Constructing a valid definition tree step by step	WorkflowBuilder, a director per scenario (e.g. PatchReleaseDirector)
State	Work item lifecycle: valid operations depend on state	WorkItemState → Created, Assigned, InProgress, Completed, Rejected...
Strategy	Assignment and routing rules differ per organisation	AssignmentStrategy → RoleBased, LeastWorkload, RuleBased
Chain of Responsibility	Multi-level approval and escalation	ApprovalHandler chain; unanswered handlers escalate upward
Command	Actions on work items, recorded for audit and undo	AssignCommand, CompleteCommand, RejectCommand, CommandHistory
Observer	Many parties react to the same workflow event	WorkflowEventSource, observers such as AuditLog, Notifier, DeadlineMonitor
Adapter	External systems with foreign interfaces	ExternalSystem target, SteamCertAdapter, CustomsApiAdapter
Decorator	Tasks gain validation, priority, audit or security without new subclasses	TaskDecorator → PriorityTask, SecuredTask, ValidatedTask

Tier 2

Pattern	Problem	Notes
Iterator	Traversing the composite without exposing storage	Depth-first, or "available items only" iterators
Memento	Rollback of instance state	Pairs naturally with Command for undo
Mediator	Completion of one item activating dependents	WorkflowCoordinator, so tasks don't reference siblings
Facade	Simple entry point for scenarios/main	TividyEngine: deploy(), start(), complete(), report()

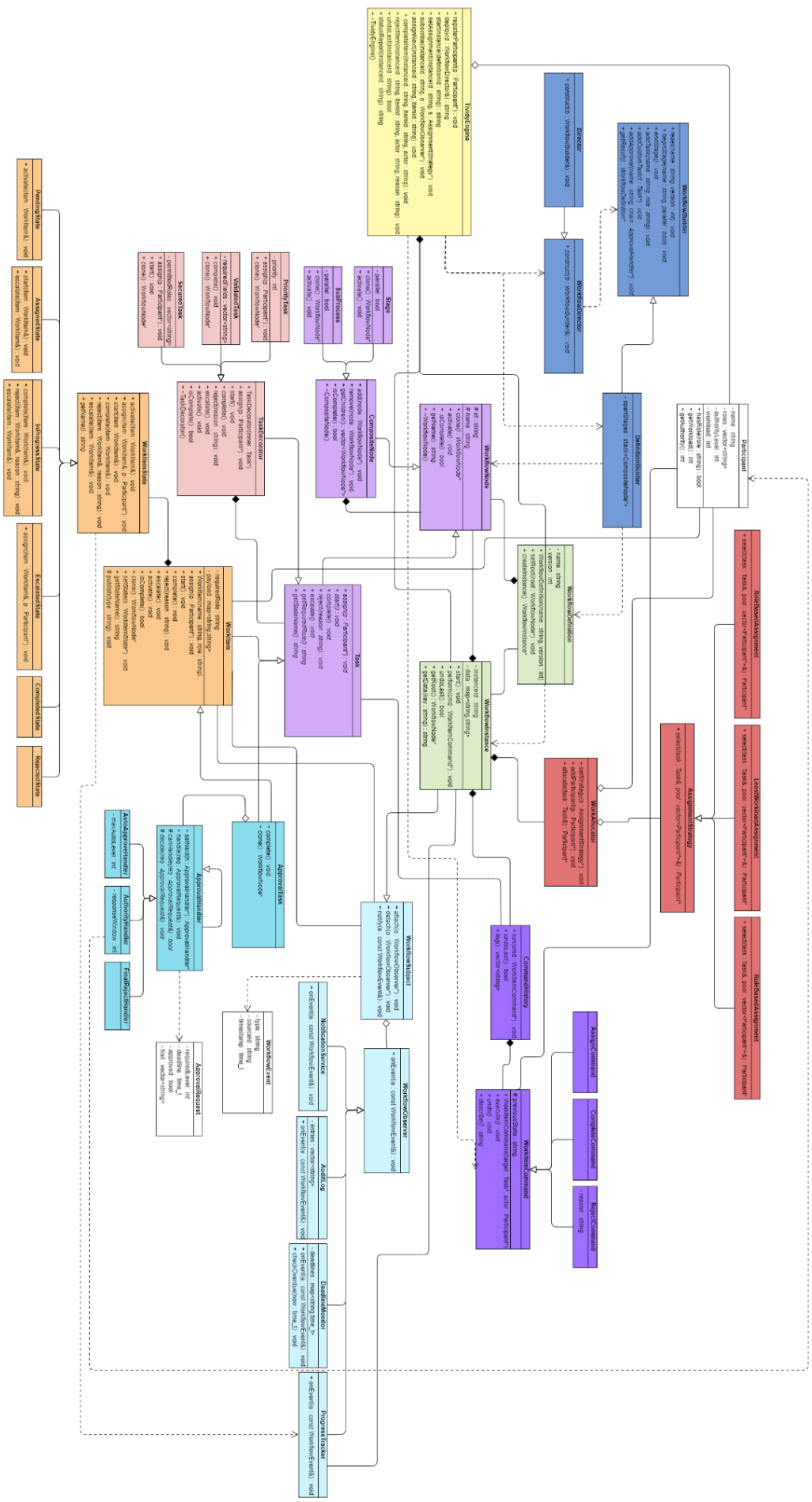
Unsafe patterns to drop:

- Composite
- State
- Prototype

Class UML

VP link:

<https://online.visual-paradigm.com/share.jsp?id=343138303733392d39>



Activity Diagram 1:

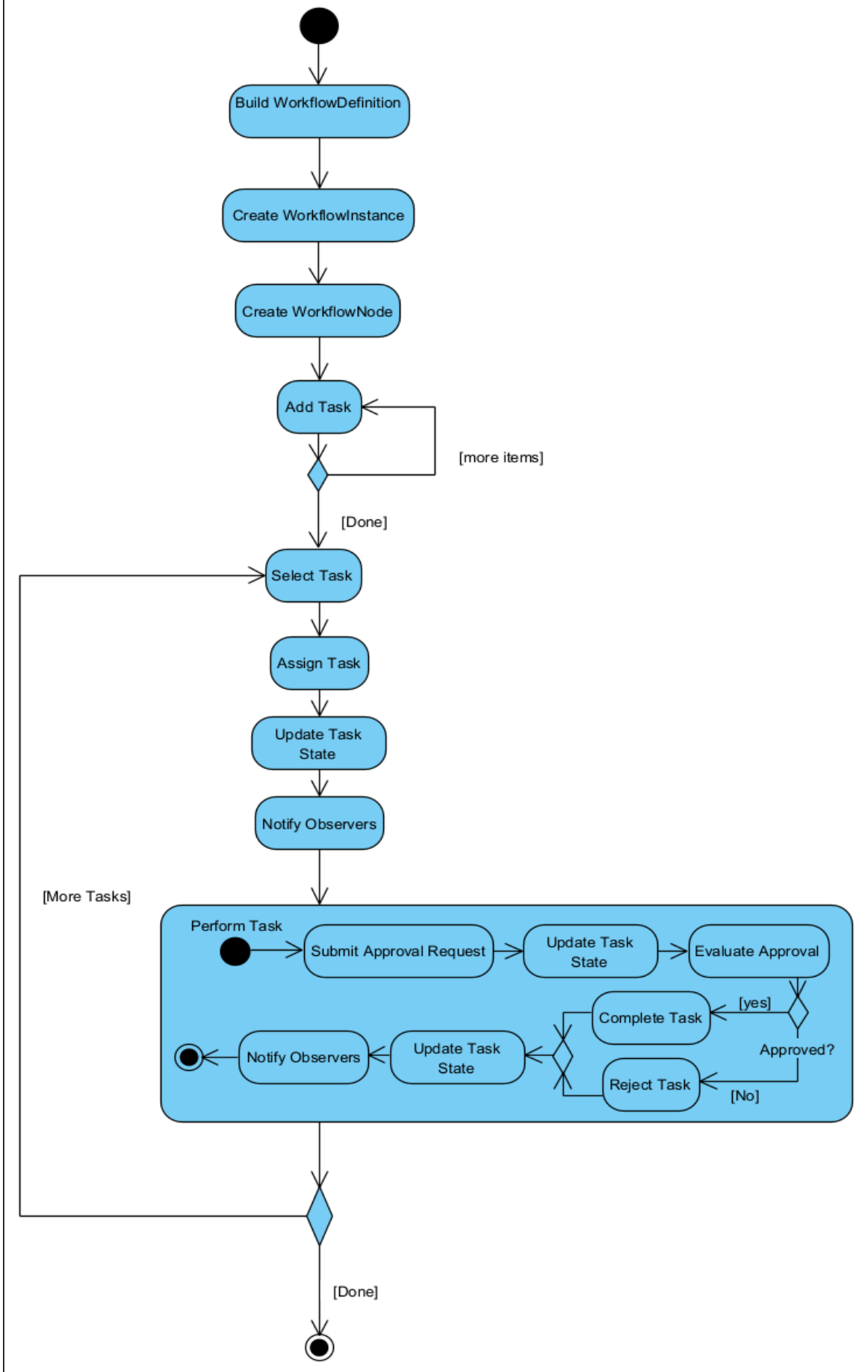
Generic Workflow from creation to completion

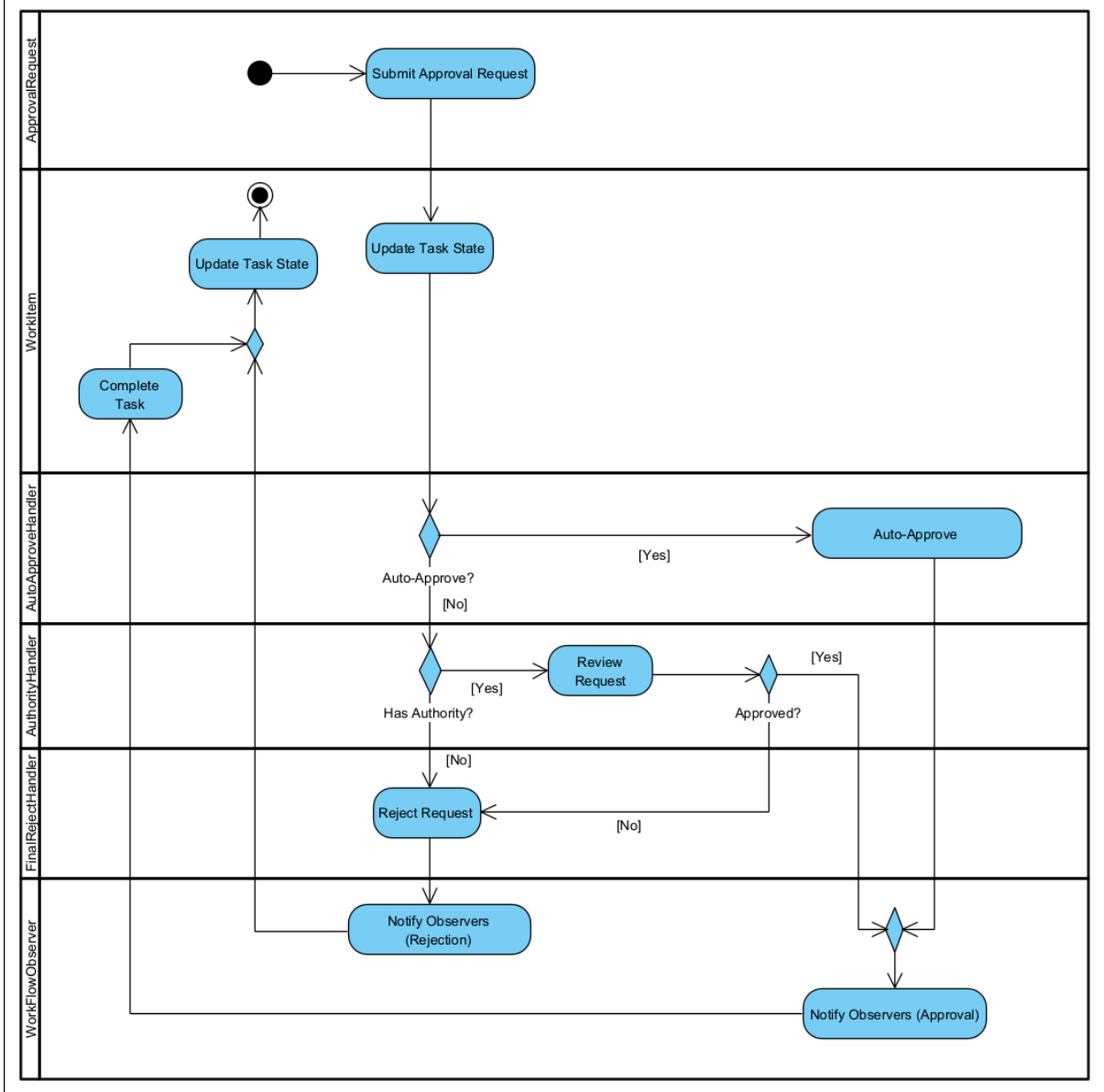
Activity Diagram 2:

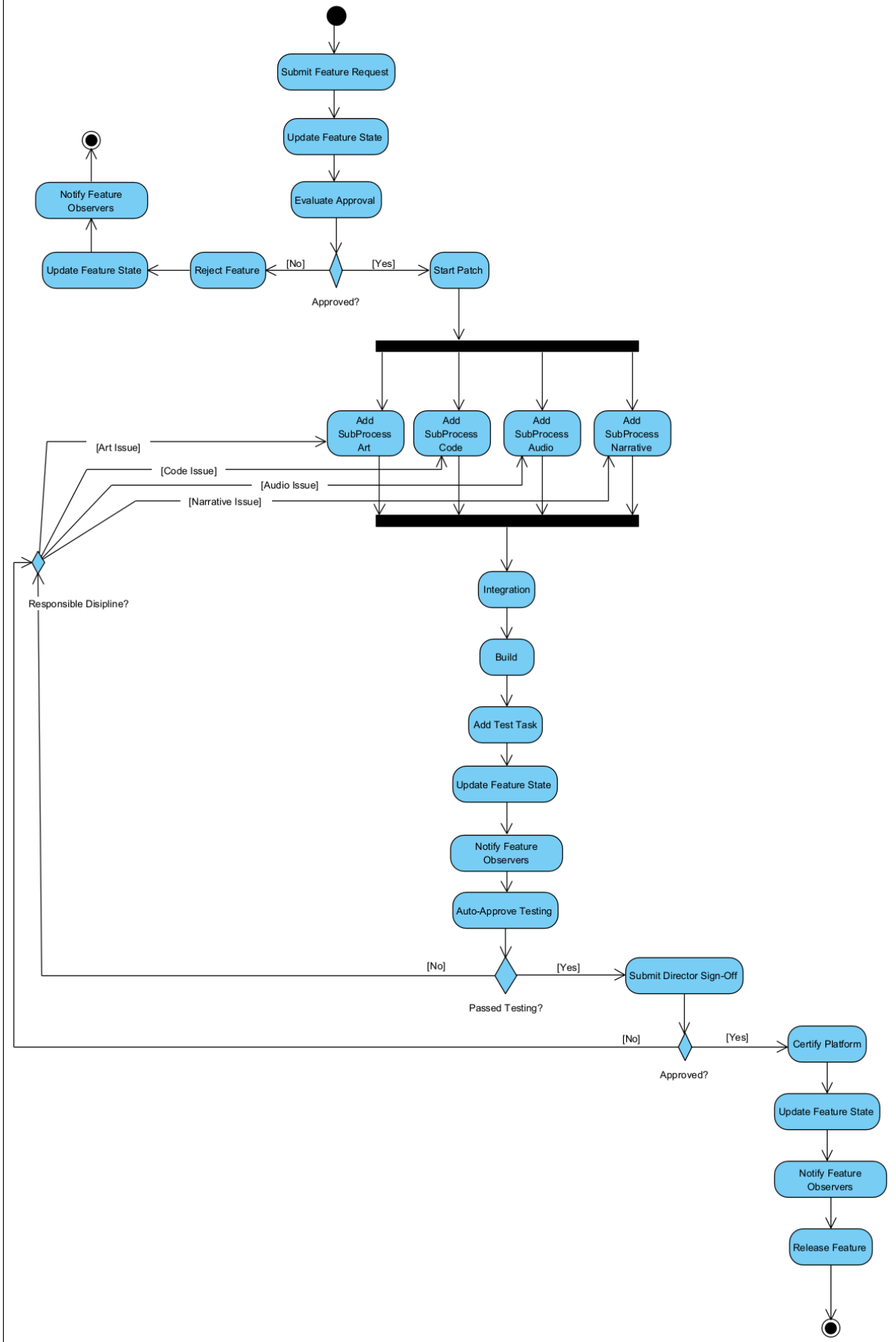
TIVIDY Approval Workflow

Activity Diagram 3:

Scenario A Workflow







Sequence Diagram 1:

Completing a work item.

Sequence Diagram 2:

Determining what happens next.

State Diagram 1:

WorkItemState.

